NovaOps v2 — Internal Engineering Notes

> Written for an engineer joining the project cold.
> Covers every major subsystem, how they connect, and why decisions were made.

1. What Is This System?

NovaOps v2 is an **autonomous SRE incident resolution system**. When a production alert fires (via PagerDuty or direct webhook), it:

1. Investigates the incident using a multi-agent AI pipeline
2. Proposes a remediation action (rollback, scale, restart, or escalate)
3. Independently validates that proposal with a second AI panel
4. Compares both conclusions and adjusts confidence accordingly
5. Routes through a policy engine — either auto-executes or notifies a human for approval

The LLM backend is **Amazon Nova 2 Lite** (or Nova Pro for the Jury) on AWS Bedrock.
The orchestration framework for the War Room is **AWS Strands** (GraphBuilder DAG).
The API layer is **FastAPI**.

2. High-Level Flow — 3 Stages

```
PagerDuty Alert
       │
       ▼
┌──────────────────────────────────────────────────────────────┐
│  FastAPI  POST /webhook/pagerduty                            │
│  Returns immediately. Pipeline runs as a background task.    │
└──────────────────────────────────────────────────────────────┘
                          │
                          │
              ┌───────────▼──────────┐
              │  CONTEXT AGGREGATORS │  ← shared by BOTH pipelines
              │  logs / metrics /    │
              │  k8s / github        │
              └──────────────────────┘
                     │          │
                     │          │
            full ctx │          │ raw ctx only
                     ▼          ▼
         ┌──────────────────┐  ┌──────────────────────────┐
         │   STAGE 1        │  │   STAGE 2                │
         │   WAR ROOM       │  │   JURY VALIDATION        │
         │   (Strands DAG)  │  │   (Independent Panel)    │
         └──────────────────┘  └──────────────────────────┘
                │ proposed_action          │ verdict + confidence
                └──────────────────────────┘
                             │
                             ▼
              ┌──────────────────────────┐
              │   STAGE 3                │
              │   CONVERGENCE CHECK      │
              │   agree? → +0.15 conf    │
              │   disagree? → -0.30      │
              └──────────────────────────┘
                             │
                             ▼
              ┌──────────────────────────┐
              │   GOVERNANCE GATE        │
              │   risk score + policy    │
              │   → ALLOW_AUTO           │
              │   → REQUIRE_APPROVAL     │
              └──────────────────────────┘
                             │
                             │
              ┌──────────────┴──────────────┐
              ▼                             ▼
     Auto-execute                Slack notification
     (Executor)                  → human approves
                                 → Executor
```

Key design principle: The Jury receives ONLY raw incident context — never the War Room's reasoning. This prevents groupthink. Two pipelines reaching the same conclusion independently is a strong signal; disagreement triggers mandatory human review.

3. Stage 1 — War Room (AWS Strands GraphBuilder)

Entry Point

- CLI: `python -m agents "P2 OOM alert on payment-service"`
- API: called from `api/server.py` background task
- Code: `agents/main.py` → `agents/graph.py`

Agent Graph (DAG)

```
              [triage]
             /   |   \   \
[log_analyst] [metrics] [k8s] [github]   ← parallel
             \   |   /   /
          [root_cause_reasoner]
                  |
             [critic] ─── FAIL (max 3 loops) ──→ back to analysts
                  |
              PASS
                  |
       [remediation_planner]
```

Agents and Their Roles

Agent	Model tier	Input	Output schema
`triage`	LOW	Raw alert text	`TriageOutput` (domain, severity, service)
`log_analyst`	MED	Triage + logs	`AnalystFindings`
`metrics_analyst`	MED	Triage + metrics	`AnalystFindings`
`k8s_inspector`	MED	Triage + K8s events	`AnalystFindings`
`github_analyst`	MED	Triage + commits	`AnalystFindings`
`root_cause_reasoner`	HIGH	All 4 analyst outputs	`RootCauseOutput` (ranked hypotheses)
`critic`	LOW	All above	`CriticOutput` (PASS/FAIL verdict)
`remediation_planner`	MED	All above	`RemediationPlan` (action + params)

Critic Loop

- Critic outputs `{"verdict": "PASS" or "FAIL", "confidence": float, ...}`
- If FAIL → graph loops back to all 4 analysts for another round
- Max 3 loops enforced by `MAX_REFLECTION_LOOPS = 3`
- After 3 loops without PASS → proceeds anyway with escalation bias
- Code: `agents/graph.py` → `_critic_passed()` / `_critic_failed()`

Domain Classification

- Alert text is classified into 1 of 6 domains before the graph runs
- Domains: `oom`, `traffic_surge`, `deadlock`, `config_drift`, `dependency_failure`, `cascading_failure`
- Code: `agents/skills.py` → `classify_domain()`
- Domain determines which skill playbooks are injected into agent prompts

Mock Mode

- `NOVAOPS_USE_MOCK=1` (default) → no Bedrock calls, uses `_MockGraph`
- Mock returns pre-baked JSON for `oom` and `traffic_surge` domains
- All other domains return a safe `noop_require_human` with low confidence

Output Artifacts (per incident)

Written to `plans/{incident_id}/`:

- `report.md` — full investigation narrative
- `structured.json` — all parsed agent outputs
- `validation.json` — per-agent schema validation scores
- `findings/*.json` — individual agent outputs
- `plan.md` — investigation plan (updated to COMPLETED)
- `trace.json` — only written on failure (stack trace + error)

4. Stage 2 — Jury Validation (Independent Blind Panel)

Entry Point

Called from `agents/main.py` after War Room completes:

```
jury = JuryOrchestrator(mock_sensors=..., mock_llm=...)
jury_result = jury.run(alert_name=..., service_name=..., namespace=...)
```

Code: `Agent_Jury/jury_orchestrator.py`

Jury Workflow

```
OBSERVE   → 4 aggregators fetch raw context (fresh, independent)
               |
DELIBERATE → 4 jurors analyse IN PARALLEL
               |
JUDGE      → Judge LLM synthesises 4 verdicts into 1 binding verdict
               |
GATE       → EscalationGate runs 6 safety checks
               |
           → returns { proposed_action, confidence, should_escalate }
```

The 4 Jurors

Juror	File	Specialisation	Mock confidence
Log Analyst	`jurors/log_analyst.py`	Error logs, OOM, crash traces, exception patterns	0.88
Infra Specialist	`jurors/infra_specialist.py`	K8s pod health, CPU/memory, OOMKilled, HPA scaling	0.91
Deployment Specialist	`jurors/deployment_specialist.py`	Git commit timing, config regressions, runbook correlation	0.85
Anomaly Specialist	`jurors/anomaly_specialist.py`	External APIs, DNS, DB, security — 3-layer detection	0.10 (no anomaly)

Each juror extends `BaseJuror` (`jurors/base_juror.py`), which handles LLM invocation, JSON parsing, and failure verdicts. All jurors use the `NovaClient` (`agent/nova_client.py`) which talks to Amazon Bedrock directly via boto3.

Anomaly Specialist — 3-Layer Detection

This juror is the "catch-all" for patterns the other three miss:

```
Layer 1 — RAG Signal
  TF-IDF score vs runbook index.
  Low score = no known runbook = novel/first-seen error.
  Signal: "HIGH probability of unknown incident type"

Layer 2 — Pattern Scan (always runs, no LLM needed)
  Regex across ALL raw context text.
  4 categories: External API, Network/DNS, Database, Security/Auth
  If matched → confidence boosted to ≥ 0.6
  If matched → detected_categories[] populated

Layer 3 — LLM Residual (full context to Nova)
  "What anomalous factor, if any, is driving this?"
  Returns verdict, confidence, reasoning
```

Pattern examples scanned:

- External API: `connection refused`, `502 bad gateway`, `ECONNRESET`
- DNS: `DNS resolution failed`, `no such host`, `UnknownHostException`
- DB: `deadlock`, `connection pool exhausted`, `ORA-\d+`, `redis.*timeout`
- Security: `401 unauthorized`, `JWT.*invalid`, `SSL.*handshake`

Judge LLM

- File: `Agent_Jury/judge.py`
- Uses `NovaClient` with system prompt as "Chief Judge"
- Receives all 4 juror verdicts with their confidence scores
- Outputs: `final_root_cause`, `judge_reasoning`, `proposed_action`, `confidence`, `dissenting_jurors`
- In mock mode: defers to the highest-confidence juror

Escalation Gate (6 Safety Checks)

File: `Agent_Jury/escalation_gate.py`

Check	Condition	Triggers escalation when
1	Pipeline failed	`agent_status == "failed"`
2	Judge low confidence	`judge_confidence < 0.6`
3	LLM failure	Any juror has `confidence == 0.0`
4	Juror disagreement	High-confidence jurors recommend different actions
5	Anomaly detected	`detected_categories[]` is non-empty
6	Judge chose noop	`proposed_action.tool == "noop_require_human"`

Any check firing → `should_escalate = True` → passed to Convergence Check.

5. Stage 3 — Convergence Check

File: `pipeline/convergence.py`

Compares the War Room's proposed action vs the Jury's proposed action.

```
base_confidence = war_room.critic.confidence
              OR  war_room.root_cause.confidence_overall
              OR  top_hypothesis.confidence
              OR  0.5 (fallback)

AGREE = (war_room_action == jury_action) AND NOT jury_should_escalate

AGREE:
  adjusted = min(1.0, base + 0.15)
  source = "convergence_agreement"

DISAGREE:
  adjusted = max(0.0, base - 0.30)
  source = "convergence_disagreement"
```

The `adjusted_confidence` and full summary are:

1. Logged to `audit.jsonl` as event `CONVERGENCE_CHECK`
2. Passed into the Governance Gate via `incident["convergence"]`

6. Governance Gate

File: `governance/gate.py`

This is the single control plane for all execution. Nothing runs outside it.

Two Entry Points

`evaluate()` — called automatically at end of every investigation:

1. Reads `convergence.adjusted_confidence` if convergence ran
2. Logs pipeline events (TRIAGE_COMPLETE, HYPOTHESIS_FORMED, CRITIC_VERDICT)
3. Runs `PolicyEngine.evaluate(ctx)` → `PolicyDecision`
4. Logs `GOVERNANCE_DECISION` to audit trail
5. If `ALLOW_AUTO` → calls `executor.execute()` immediately
6. If `DENY` → logs `EXECUTION_DENIED`
7. Saves `governance.json`

`approve_and_execute()` — called from `POST /api/incidents/{id}/approve` :

1. Loads persisted `governance.json`
2. Guards: DENY, already executed → rejects
3. Logs `HUMAN_OVERRIDE`
4. Calls `executor.execute()`
5. Updates `governance.json` with approval outcome

Risk Score Formula (0–100)

```
risk = action_weight + severity_weight + confidence_penalty

action_weight:
  rollback_deployment = 40
  restart_pods        = 20
  scale_deployment    = 15
  noop_require_human  = 0

severity_weight:
  P1 = 30, P2 = 20, P3 = 10, P4 = 5

confidence_penalty (when confidence < 0.75):
  penalty = int((0.75 - confidence) * 40)
```

Policy Rules (first match wins)

Policy	Condition	Decision
`noop_requires_approval`	action = noop	REQUIRE_APPROVAL
`p1_always_requires_approval`	severity = P1	REQUIRE_APPROVAL
`rollback_always_requires_approval`	action = rollback	REQUIRE_APPROVAL
`low_confidence_escalate`	confidence < 0.65	REQUIRE_APPROVAL
`high_confidence_p3_p4_auto`	P3/P4 + restart/scale + conf ≥ 0.75	ALLOW_AUTO
`p2_scale_high_confidence`	P2 + scale + conf ≥ 0.85	ALLOW_AUTO
`default_require_approval`	(catch-all)	REQUIRE_APPROVAL

Confidence precedence (for governance):

`convergence.adjusted_confidence > critic.confidence > root_cause.confidence_overall > top_hypothesis.confidence > 0.0`

The convergence-adjusted value always wins when it exists. This means disagreement between War Room and Jury will raise the risk score and push toward REQUIRE_APPROVAL regardless of what the policy might say for action/severity alone.

Artifacts

- `governance.json` — full decision record with risk score, policy, status
- `governance_report.md` — human-readable summary
- `audit.jsonl` — append-only event log (never overwritten)

7. Context Aggregators

Directory: `aggregator/`
All 4 aggregators have `use_mock=True` default. In live mode they call real AWS/GitHub APIs.

Module	Real API	Mock data
`logs.py`	AWS CloudWatch `filter_log_events`	4 OOM log lines
`metrics.py`	Prometheus queries	CPU/memory dict
`kubernetes_state.py`	Kubernetes API pod events	OOMKilled events
`github_history.py`	GitHub REST API	Recent commits list

Both Stage 1 (War Room) and Stage 2 (Jury) call these independently. The Jury always does a fresh fetch so results can differ from the War Room's fetch if live.

8. Knowledge Base / RAG

File: `agents/knowledge_base.py`

- **TF-IDF** based — no external vector DB, no dependencies
- Indexes all `.md` files in `runbooks/` directory at startup
- Query: alert text → scored against all runbook terms → returns best match content
- Used in Stage 1 (injected into agent prompts via `retrieve_knowledge` tool)
- Used in Stage 2 (injected into Jury context, and as Layer 1 signal for Anomaly Specialist)
- **Self-learning**: When a PIR (Post-Incident Report) is generated, it can be saved as a new runbook → future incidents benefit from past learnings
- Bedrock Knowledge Bases (managed RAG) optionally supported via `USE_BEDROCK_KB=true`

9. Skills / Domain Playbooks

Directory: `skills/`

Domain-specific playbooks injected into agent system prompts at graph-build time. Three types per domain:

```
skills/domains/{domain}/
  SKILL.md        → domain overview (injected into triage prompt)
  analyst.md      → analyst-specific investigation steps
  remediation.md  → remediation-specific decision guidance
```

```
skills/_shared/
  triage/SKILL.md     → general triage procedure (all domains)
  escalation/SKILL.md → when and how to escalate
```

Covered domains: `oom` , `traffic_surge` , `deadlock` , `config_drift` , `dependency_failure` , `cascading_failure`

The `classify_domain()` function in `agents/skills.py` does keyword matching on the alert text to select the right domain before the graph is built. This controls which playbook context each agent receives.

10. Typed Schemas

File: `agents/schemas.py`

Every agent in the War Room produces structured JSON. These dataclasses parse and validate it:

Schema	Parses	Key fields
`TriageOutput`	triage node	domain, severity, service_name, namespace
`AnalystFindings`	4 analyst nodes	findings dict, hypothesis_support
`RootCauseOutput`	root_cause_reasoner	hypotheses[], confidence_overall
`Hypothesis`	inside RootCauseOutput	rank, description, confidence, recommended_action
`CriticOutput`	critic node	verdict (PASS/FAIL), confidence, feedback
`RemediationPlan`	remediation_planner	action_taken, parameters, justification
`WarRoomResult`	all nodes combined	triage, analysts, root_cause, critic, remediation

Parsing strategy: tries `json.loads()` first, strips markdown fences, falls back to scanning for `{` positions. Invalid outputs return safe empty defaults instead of crashing.

11. API Server

File: `api/server.py`
Framework: FastAPI
Start: `uvicorn api.server:app --reload`

Endpoints

Method	Path	Purpose
POST	`/webhook/pagerduty`	Receive alert, start 3-stage pipeline as background task
GET	`/api/incidents`	Last 50 incidents from SQLite history

Method	Path	Purpose
GET	`/api/incidents/{id}`	Single incident details
POST	`/api/incidents/{id}/approve`	Ghost Mode: human approves → GovernanceGate → Executor
POST	`/api/incidents/{id}/report`	Generate Post-Incident Report (PIR)
GET	`/api/incidents/{id}/report`	Retrieve stored PIR
GET	`/api/governance/{id}/decision`	Governance decision + risk score JSON
GET	`/api/governance/{id}/audit`	Full append-only audit trail
GET	`/health`	Liveness probe

The `/approve` endpoint is the only way to trigger human-approved execution. It routes exclusively through `GovernanceGate.approve_and_execute()` — no executor call exists anywhere else in the server.

Supporting Modules

- `api/history_db.py` — SQLite via built-in `sqlite3`. Table: `incidents`. Stores: id, service, domain, severity, analysis, proposed action, status, report path, PIR.
- `api/slack_notifier.py` — Posts Slack messages via webhook URL. In mock mode prints to stdout.

12. Tool Executor

File: `tools/executor.py`

The `RemediationExecutor` is the final safety checkpoint before any real action:

1. Checks `tool` is in `EXECUTABLE_TOOLS = {rollback_deployment, scale_deployment, restart_pods}`
2. Validates `service_name` matches `^[a-z0-9][a-z0-9-]{0,62}$`
3. Validates `namespace` matches same pattern
4. Dispatches to `KubernetesActions` (`tools/k8s_actions.py`)

`KubernetesActions` wraps real `kubectl` / K8s API calls. In mock mode it simulates success responses.

`noop_require_human` is **not** in `EXECUTABLE_TOOLS` — it can never be auto-executed.

13. Audit Trail

File: `governance/audit_log.py`

- Location: `plans/{incident_id}/audit.jsonl`
- **Append-only**: never truncated or overwritten
- Each entry: `{"event": ..., "actor": ..., "timestamp": ..., "data": {...}}`

Event Sequence (normal run)

```
ALERT_RECEIVED        → webhook received
TRIAGE_COMPLETE       → domain, severity, service classified
HYPOTHESIS_FORMED     → root cause ranked
CRITIC_VERDICT        → adversarial review result
CONVERGENCE_CHECK     → War Room vs Jury compared
GOVERNANCE_DECISION   → policy evaluated, risk scored
EXECUTION_STARTED     → tool begins (actor: SYSTEM or HUMAN)
EXECUTION_COMPLETE    → tool result logged
HUMAN_OVERRIDE        → manual approval (actor: HUMAN)
```

14. Evaluation Harness

Directory: `evaluation/`
Run: `python -m evaluation --all`

- 15 pre-defined scenarios across 6 failure domains
- Each scenario provides a synthetic alert text and expected domain/action
- Runner calls `agents/main.run()` in mock mode and checks outputs
- Useful for regression testing after prompt or schema changes

```
python -m evaluation --list          # show all 15 scenarios
python -m evaluation --scenario 3    # run one
python -m evaluation --domain oom    # run all OOM
python -m evaluation --all           # full suite
```

15. Tests

Directory: `tests/`
Run: `python -m unittest discover -s tests -v`
Count: **37 unit tests**, runs < 1s (all mock mode, no Bedrock)

Key test files:

- `test_governance_gate_logic.py` — policy decisions, risk scores, convergence confidence
- `test_graph_logic.py` — mock graph, critic loop, domain classification
- `test_schemas_logic.py` — all schema parsing with valid/invalid JSON
- `test_artifacts_logic.py` — file persistence
- `test_api_logic.py` — endpoint routing and status codes

16. Environment Variables

Variable	Default	Effect
`NOVAOPS_USE_MOCK`	`true`	Offline mode for War Room AND Jury. No Bedrock spend.
`NOVA_MODEL_ID`	`us.amazon.nova-2-lite-v1:0`	War Room Bedrock model
`AWS_DEFAULT_REGION`	`us-east-1`	Bedrock region
`AWS_BEARER_TOKEN_BEDROCK`	—	Bearer token for Bedrock access
`HACKATHON_MODE`	`false`	Alias for mock mode
`SLACK_WEBHOOK_URL`	—	Enables real Slack notifications
`USE_BEDROCK_KB`	`false`	Use managed Bedrock Knowledge Bases instead of TF-IDF

17. Key Files Quick-Reference

File	What it does
`agents/main.py`	Top-level orchestrator: Stage 1 → Stage 2 → Stage 3 → Gate
`agents/graph.py`	Builds Strands DAG, defines critic loop logic, _MockGraph
`agents/schemas.py`	All typed dataclasses, parse functions, WarRoomResult
`agents/prompts.py`	System prompts for every War Room agent
`agents/skills.py`	Domain classification, playbook loading
`agents/knowledge_base.py`	TF-IDF RAG over runbooks/
`agents/artifacts.py`	Persists all files under plans/{id}/
`agents/models.py`	BedrockModel abstraction with mock toggle
`Agent_Jury/jury_orchestrator.py`	Jury entry: OBSERVE → DELIBERATE → JUDGE → GATE
`Agent_Jury/judge.py`	Chief Judge LLM synthesis of 4 juror verdicts
`Agent_Jury/escalation_gate.py`	6-check safety gate
`Agent_Jury/jurors/base_juror.py`	Base class: LLM call, JSON parse, failure verdict
`Agent_Jury/jurors/anomaly_specialist.py`	3-layer anomaly detection (RAG + regex + LLM)
`agent/nova_client.py`	boto3 wrapper for Amazon Nova (used by Jury)
`pipeline/convergence.py`	Compares War Room vs Jury, adjusts confidence

File	What it does
`governance/gate.py`	GovernanceGate: evaluate + approve_and_execute
`governance/policy_engine.py`	YAML policy loader, risk scorer, PolicyDecision
`governance/audit_log.py`	Append-only JSONL event writer
`governance/report.py`	Generates governance_report.md
`tools/executor.py`	RemediationExecutor: validates + dispatches to K8s
`tools/k8s_actions.py`	rollback_deployment, scale_deployment, restart_pods
`api/server.py`	FastAPI: webhook, incidents, approve, governance, audit
`api/history_db.py`	SQLite incident log
`api/slack_notifier.py`	Slack webhook notifications
`aggregator/logs.py`	CloudWatch ERROR/FATAL log fetcher
`aggregator/metrics.py`	Prometheus CPU/memory queries
`aggregator/kubernetes_state.py`	K8s pod events and health
`aggregator/github_history.py`	GitHub commit history

18. Data Flow Diagram — Confidence Through the System

```
War Room runs
  └─→ triage.severity ──────────────────────────────────────┐
  └─→ critic.confidence  ──────────────────────────────────┐  |
  └─→ root_cause.confidence_overall (backup) ──────────────┐|  |
  └─→ top_hypothesis.confidence (backup)                  ||  |
                                                          ||  |
Jury runs                                                 ||  |
  └─→ judge_verdict.confidence ──────────────┐            ||  |
  └─→ should_escalate ───────────────────────┤            ||  |
                                             ▼            ||  |
                                 convergence.check_convergence()
                                   base = critic.confidence  ||  |
                                   AGREE  → +0.15  ←────────┘|  |
                                   DISAGREE → -0.30           |  |
                                            |                 |  |
                                            ▼ adjusted_conf   |  |
                                 governance.gate.evaluate()   |  |
                                   ctx.confidence = adj_conf  |  |
                                   ctx.severity ←─────────────┘  |
                                   risk_score = action_w          |
                                              + severity_w ←──────┘
                                              + conf_penalty
                                            |
                                       PolicyEngine
                                       first matching rule
                                            |
                                  ALLOW_AUTO | REQUIRE_APPROVAL
```

19. Rollout Phases

Phase	Behaviour	Qualifying Condition
Shadow Mode	All tools return "Simulated Success". Both pipelines post to Slack. Engineers verify manually.	Always safe to enable
Read-Only	Agents can autonomously query logs. Mitigations still require human approval.	After shadow validation
Full Autonomy	Auto-execute without human.	P3/P4 + restart/scale + `adjusted_confidence ≥ 0.75`

20. Common Gotchas

1. **Mock mode is on by default.** You WILL get plausible-looking responses with no Bedrock calls. Set `NOVAOPS_USE_MOCK=0` for live mode.
2. **Critic loop max 3.** After 3 FAIL verdicts the system proceeds anyway with escalation bias. Check logs for `"Max reflection loops reached"` .
3. **Jury independence is structural.** The Jury instantiates its own aggregators and fetches fresh context. It never reads `war_room` object internals — only the caller (`main.py`) can compare them.
4. **`noop_require_human` can never be auto-executed.** Even if governance says `ALLOW_AUTO` , gate.py overrides it: `noop_require_human` is always `REQUIRE_APPROVAL` .
5. **Convergence confidence source wins.** If convergence ran, `ctx.confidence` in GovernanceContext is always the convergence-adjusted value, regardless of what critic or root_cause returned.
6. **audit.jsonl is append-only.** Never truncate it. Load with `AuditLog(incident_id).read_all()` .
7. **All agent outputs are parsed defensively.** Malformed JSON from LLM is handled gracefully — falls back to safe defaults. Check `validation.json` to see which nodes produced valid schema-compliant output.
8. **The Anomaly Specialist confidence inversion.** Low confidence (e.g. 0.1) means "nothing anomalous detected" — which is useful signal. High confidence means something external is wrong. Don't mistake low confidence for failure.
9. **PIRs become runbooks.** When you generate a PIR for an incident, calling `knowledge_base.save_as_runbook()` makes that incident's learnings available to future TF-IDF matches.
10. **Plans directory is gitignored.** All investigation artifacts are local-only. They live in `plans/{incident_id}/` .